

PAD (PPGCC) - Programação OpenMP (2025)

Grupo de até 2 pessoas

Instruções: O desempenho pode ser medido no próprio equipamento pessoal/profissional, ou em equipamento institucional.

Ao se utilizar o equipamento pessoal, deve-se especificar o equipamento utilizado, citando quantidade de memória principal, codinome do processador, destacando a quantidade de núcleos reais e virtuais.

A atividade pode ser realizado em **grupo de até duas pessoas**, de duas formas/opções possíveis:

Opção 1: Trazer código-fonte de interesse

Isolar e preparar um trecho (considerado pequeno) de um código-fonte de interesse (para projeto de pesquisa da pós-graduação) em C/C++ ou Fortran, com seu respectivo conjunto de dados de entrada, cuja execução gaste, pelo menos, na casa de dezenas de segundos. Otimizar o código por OpenMP e fazer medidas de desempenho relacionadas a: tempo de execução e *speedup*.

A entrega da opção 1 consiste em um **arquivo compactado** contendo:

- A. Dois ou mais **arquivos relacionados ao código-fonte utilizado, o conjunto de dados** de entrada e,
- B. Um relatório que deve conter uma breve descrição sobre o código-fonte utilizado, explicando para qual finalidade ele foi criado, e os dados das medidas de desempenho citadas.

Opção 2: Atividades práticas de programação

1. Construir um programa com vetorização e bloqueio para utilização efetiva de cache (quando possível), utilizando programação paralela por *OpenMP* para multiplicar duas matrizes (*arrays* 2D) quadradas de $N \times N$ elementos, onde $N=10^3$ e 10^4 de números do tipo *float*.

Executar e variar a quantidade de *threads* da seguinte forma: 1, 2, 4 e, se possível, mais.

Resultados requisitados no relatório:

- a. Mostre um pequeno teste com valores conhecidos dispostos em um vetor de poucos elementos, provando que o algoritmo desenvolvido funciona para duas ou mais *threads*.
- b. Analise e coloque em um gráfico os tempos de processamento, *Speedup*, Eficiência e *MFlops* obtidos, variando-se o número de *threads* conforme pedido, bem como a utilização da memória *Cache* através do PAPI, ou seja, informe a quantidade de *Cache Misses* para cada simulação.

2. Christian Goldbach (1690-1764) foi um matemático nascido na Prússia, contemporâneo de Euler. Uma das suas mais famosas conjecturas ainda não devidamente provadas estabelece que todo número par maior que dois é a soma de dois números primos, por exemplo, $28 = 5 + 23$.

O programa disponibilizado calcula a primeira ocorrência de cada soma de dois primos encontrados para números que variam de 2 até 32000.

Paraleliza-o usando *OpenMP* introduzindo a cláusula "*for*" no primeiro laço (linha 19), usando a cláusula *Schedule* para modificar as diferentes formas de balanceamento de carga entre os processadores:

- a. *STATIC*, variar tamanho do *chunk*
- b. *DYNAMIC*
- c. *GUIDED*

Compare os tempos de processamento com número de *threads* similar ao exercício 1. Compare ainda com o uso do laço paralelo sem nenhuma cláusula que mude o escalonamento.

3. Crie um programa paralelo que necessariamente utilizar TASKs em OpenMP para:
- a. Cálculo da série de Fibonnaci de forma recursiva seguindo o código modelo mostrado abaixo (o programa serial deve demandar da ordem de alguns segundos de execução) e verifique o speedup para 1, 2, 4, 8, 16 *threads*:

```
int main(int argc, char* argv[]) {
    [...]
    fib(input);
    [...] }

int fib(int n) {
    if (n < 2) return n;
    int x = fib(n - 1);
    int y = fib(n - 2);
    return x+y; }
```

- b. modifique o programa para evitar a criação de *TASKs* (use a cláusula *IF*) quando o valor de n for menor que 30 ($n < 30$) e recalcule o *speedup*
- c. modifique novamente o código para que o cálculo da série de Fibonacci seja **não-recursivo** quando $n < 30$. Recalcule o *speedup*.